

5G Millisecond-Resolution Network Benchmarking Dataset & Time Series Foundation Models

Paper: "msData: A Millisecond-Resolution Network Dataset for Advancing Time Series Foundation Models"

This repository contains code and data for benchmarking shallow models, transformer-based deep learning models, and Time Series Foundation Models (TSFMs) on high-frequency 5G network traffic forecasting tasks using millisecond-resolution measurements from an operational 5G Radio Access Network (RAN).

Overview

This work addresses a critical gap in TSFM evaluation: the lack of diverse, high-frequency datasets. While existing benchmarks focus on low-frequency time series (seconds to years), production wireless networks generate data at millisecond granularity with complex temporal dynamics. This repository provides:

1. **5G_millisecond.csv:** A novel dataset capturing millisecond-resolution RAN metrics
2. **Comprehensive benchmarks:** 12 models evaluated in both univariate and multivariate settings
3. **Supplementary notebooks:** Full experimental reproductions of main experiments

Quick Start

Main experiment dataset: The main results reported in this repository use the filtered dataset [data/filtered/youtube_static.csv](#).

1. Create the required conda environments:

```
conda env create -f environment.yml
conda env create -f arf_environment.yml
```

Use:

- tf-env for Random Forest, XGBoost, TTM, Chronos, Lag-Llama, iTransformer, and PatchTST
- river-env for ARF and OLR

2. Run the notebook-based experiments:

```
conda activate tf-env
scripts\reproduce_univariate.bat
scripts\reproduce_multivariate.bat
```

3. Run ARF and OLR separately:

```
conda activate river-env
python scripts\ARF_Univariate.py --seeds 42
python scripts\ARF_Multivariate.py --seeds 42
python scripts\OLR_Univariate.py
python scripts\OLR_Multivariate.py
```

Environment Map

- **tf-env**: Random Forest, XGBoost, Naive, TTM, Chronos, Lag-Llama, iTransformer, PatchTST
- **river-env**: Adaptive Random Forest (ARF), Online Linear Regression (OLR)

Windows Note

This repository uses Windows batch files (**.bat**) for notebook execution.

Run them from Anaconda Prompt or Command Prompt after activating the correct conda environment.

For detailed setup instructions, model-specific installation steps, and full experiment commands, see **REPRODUCIBILITY.md**.

Dataset

Raw Data: **5G_millisecond.csv**

Main experiment dataset: **youtube_static.csv** (**data/filtered/youtube_static.csv**)

The filtered dataset **youtube_static.csv** is generated from the raw dataset using:

```
python scripts/filter_data.py
```

Format: CSV (32,816 rows, 7 columns)

Size: ~1.5 MB (uncompressed)

Data Source & Collection

- **Source:** OpenRanlab 5G O-RAN testbed deployment
- **Collection Duration:** ~33 seconds of continuous measurements (January 1, 2022, 00:00:00 to 00:00:32.814)
- **Temporal Resolution:** 1 millisecond
- **Collection Method:** Software-Defined Radios (Ettus USRPs) configured as base station, collecting from multiple User Equipments (UEs)
- **Mobility Pattern:** Static
- **Traffic Class:** Video streaming (YouTube)

Forecasting Setup

Task: Short-term bitrate forecasting (1-96 steps ahead)

Horizon: 96 steps = 96 milliseconds into future

Input Sequence Length: 5 steps (50 milliseconds of history for shallow models)

Prediction Target: `mac_dl_brate` (downlink bitrate)

Univariate vs. Multivariate

Setting	Features Used
Univariate	Lagged values of <code>mac_dl_brate</code> only
Multivariate	4 features (CQI, MCS, pkt ok, pkt nok)

Models Benchmarked

Shallow Machine Learning Models

- 1. **Random Forest (RF)** - Scikit-learn, optimized for multi-step forecasting
- 2. **XGBoost (XGB)** - Gradient boosting ensemble
- 3. **Adaptive Random Forest (ARF)** - River library; handles streaming/online learning
- 4. **Online Linear Regression (OLR)** - Incremental learning baseline
- 5. **Naive Baseline** - Previous value repeated

Transformer-based Deep Learning Models

- 6. **PatchTST** - Patch-based transformer
- 7. **iTransformer** - Attention over features

Time Series Foundation Models (TSFMs)

- 8. **TTM (Tiny Time Mixers)** - Light-weight pre-trained model
- 9. **Chronos** - Probabilistic TSFM; supports zero-shot and fine-tuned variants
- 10. **Lag-Llama** - Decoder-only transformer; zero-shot and fine-tuned variants (supports univariate setting only)

Results Summary

Table: Performance Metrics (Scaled RMSE / MAE)

Model	Univariate	Multivariate
RF	0.0344 / 0.0227	0.0342 / 0.0226
XGB	0.0354 / 0.0232	0.0354 / 0.0231
ARF	0.0270 / 0.0189	0.0175 / 0.0130
Naive	0.0418 / 0.0240	0.0418 / 0.0240

Model	Univariate	Multivariate
OLR	0.0551 / 0.0308	0.0555 / 0.0310
PatchTST	0.0327 / 0.0212	0.0321 / 0.0207
iTransformer	0.0325 / 0.0208	0.0324 / 0.0208
TTM Zero-shot	0.0359 / 0.0230	0.0359 / 0.0230
TTM Fine-tuning	0.0371 / 0.0237	0.0393 / 0.0250
Chronos Zero-shot	0.0313 / 0.0185	0.0273 / 0.0181
Chronos Fine-tuning	0.0281 / 0.0178	0.0253 / 0.0176
Lag-Llama Zero-shot	0.0617 / 0.0384	-
Lag-Llama Fine-tuning	0.0474 / 0.0268	-

Repository Structure

```

tmlr-reproducibility/
├── README.md                # This file
├── DATASET.md              # Detailed dataset documentation
├── REPRODUCIBILITY.md      # Step-by-step reproduction guide
├── 5G_millisecond.csv      # Main dataset (32.8K rows, 1.5MB)
├── environment.yml         # Minimal conda dependencies
├── (TensorFlow)
├── arf_environment.yml     # ARF-specific environment
├── test_imports.py
├── scripts/
│   ├── verify_dataset.py
│   ├── filter_data.py
│   ├── ARF_Multivariate.py
│   ├── ARF_Univariate.py
│   ├── OLR_Multivariate.py
│   ├── OLR_Univariate.py
│   ├── reproduce_univariate.bat
│   ├── reproduce_multivariate.bat
│   ├── reproduce_univariate-chronos.bat
│   ├── reproduce_multivariate-chronos.bat
│   └── collect_results.py
├── notebooks/
│   ├── Random Forest_Univariate.ipynb
│   ├── XGBoost_Univariate.ipynb
│   ├── Adaptive RF - Univariate.ipynb
│   ├── OLR-Univariate.ipynb
│   ├── Naive.ipynb
│   ├── iTransformer - Univariate.ipynb
│   ├── PatchTST - Univariate.ipynb
│   ├── TTM-Univariate.ipynb
│   └── Chronos-Zeroshot - Univariate.ipynb

```

```
| | | Chronos-finetuning - Univariate.ipynb  
| | | llama-Zeroshot.ipynb  
| | | llama-Finetuning.ipynb  
| | | Random Forest_Multivariate.ipynb  
| | | XGBoost_Multivariate.ipynb  
| | | Adaptive RF - Multivariate.ipynb  
| | | OLR-Multivariate.ipynb  
| | | TTM-Multivariate.ipynb  
| | | Chronos-Zeroshot - Multivariate.ipynb  
| | | Chronos-finetuning - Multivariate.ipynb  
| | | iTransformer - Multivariate.ipynb  
| | | PatchTST - Multivariate.ipynb  
| | data/  
| | | raw/  
| | | filtered/  
| | results/  
| | | executed_notebooks/  
| | | metrics/
```

Installation

1. Environment Setup

Create the required conda environments:

```
conda env create -f environment.yml  
conda env create -f arf_environment.yml
```

TSFMs require additional one-time setup:

- Chronos
- TTM
- Lag-Llama

See REPRODUCIBILITY.md for detailed installation instructions.

2. Load Dataset

```
import pandas as pd  
  
df = pd.read_csv('data/filtered/youtube_static.csv')  
print(df.head())
```

3. Run a Simple Experiment

Univariate Random Forest:

```
jupyter notebook notebooks/Random Forest_Univariate.ipynb
```

Multivariate Adaptive Random Forest (Best Performance):

```
jupyter notebook notebooks/Adaptive RF - Multivariate.ipynb
```

4. Run All Benchmarks

See [REPRODUCIBILITY.md](#) for automated scripts and commands to reproduce the main results reported in Table 4.

Key Findings

1. **ARF Excels at High-Frequency Data:** ARF achieved the best multivariate performance (RMSE 0.0175, MAE 0.0130).
2. **TSFMs Show Promise:** Chronos fine-tuning achieved strong TSFM performance (RMSE 0.0253, MAE 0.0176).
3. **Fine-tuning Matters:** Fine-tuning improved Lag-Llama and Chronos over zero-shot baselines.

Limitations & Future Work

1. **Limited Temporal Span:** 33 seconds is short; multi-hour or multi-day traces would enable seasonality analysis.
2. **Single Traffic Class:** Video streaming only; generalization to mixed traffic unknown.
3. **Single Testbed:** Results specific to O-RAN architecture; applicability to traditional RAN/cellular unclear.

Future Work:

- Longer traces (hours/days) with realistic traffic mix
- Real-world deployment scenarios
- Anomaly detection and classification sub-tasks

Reproducibility

For complete step-by-step instructions, including environment setup, dataset preparation, model-specific setup, and experiment execution, see [REPRODUCIBILITY.md](#).

Changelog

- **v1.0** (2026-06-16): Initial release with 12 models, 32.8K samples, millisecond-resolution 5G RAN data